# Inter Processes Communication – IPC

Ms. Remyakrishnan P,
Asst. Professor, CSED,
Amrita School of Engineering

# Inter Process Communication

- Processes within a system may be ***independent*** or ***cooperating***
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience
- Cooperating processes need **inter process communication** (**IPC**)
- Two models of IPC
  - **Shared memory**
  - **Message passing**

# Communications Models

(a) shared memory (b) Message passing.

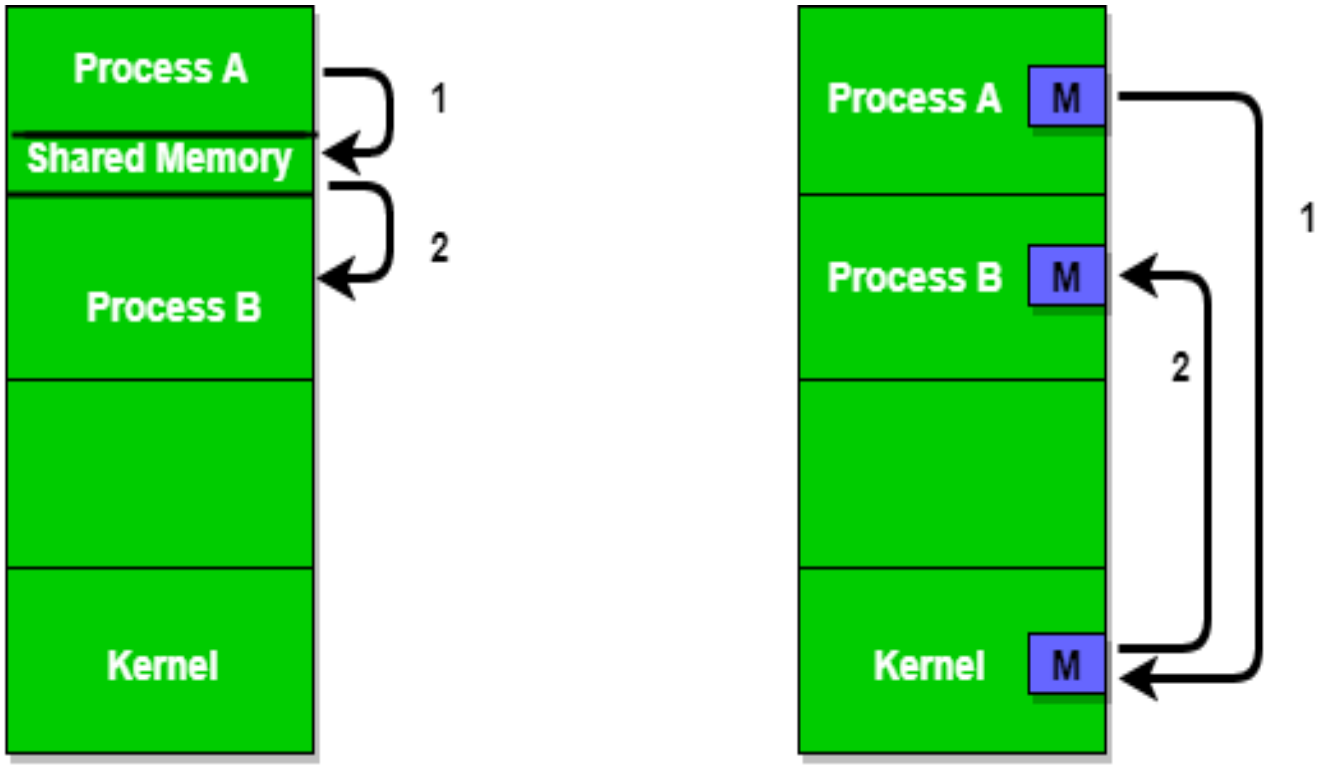



**Figure 1 -** Shared Memory and Message Passing

# Shared memory Example: Producer-Consumer Problem

- A paradigm for cooperating processes is the Producer- Consumer problem
- There are two processes: Producer and Consumer. *Producer* process produces information that is consumed by a *Consumer* process
- The two processes share a common space or memory location known as a buffer where the item produced by the Producer is stored and from which the Consumer consumes the item
- There are two versions of this problem:
  - **unbounded-buffer:** Producer can keep on producing items and there is no limit on the size of the buffer
  - **bounded-buffer:** Producer can produce up to a certain number of items before it starts waiting for Consumer to consume it.

# IPC - Shared Memory model

- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the users processes not the operating system
- Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory

# IPC – Message Passing model

- Mechanism for processes to communicate and to synchronize their actions
- Message system – processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
  - **send**(*message*)
  - **receive**(*message*)
- The *message* size is either fixed or variable

# Message Passing (Cont.)

- If processes *P* and *Q* wish to communicate, they need to:
  - Establish a ***communication link*** between them
  - Exchange messages via send/receive primitives
- A standard message can have two parts: **header and body**

  The **header part** is used for storing message type, destination id, source id, message length, and control information.
- The control information contains information like what to do if runs out of buffer space, sequence number, priority.
- Generally, message is sent using FIFO style

# Difference between Shared Memory Model and Message Passing Model in IPC :

| S.No | Shared Memory Model | Message Passing Model |
|---|---|---|
| 1. | Shared memory region is used for communication. | Message passing facility is used for communication. |
| 2. | It is used for communication between processes on a single processor or multiprocessor systems where the communicating processes reside on the same machine as the communicating processes share a common address space. | It is typically used in a distributed environment where communicating processes reside on remote machines connected through a network. |
| 3. | The code for reading and writing the data from the shared memory should be written explicitly by the Application programmer. | No such code required here as the message passing facility provides mechanism for communication and synchronization of actions performed by the communicating processes. |

# Difference between Shared Memory Model and Message Passing Model in IPC :

| S.No | Shared Memory Model | Message Passing Model |
|---|---|---|
| 4. | It provides maximum speed of computation as communication is done through shared memory so system calls are made only to establish the shared memory. | It is time consuming as message passing is implemented through kernel intervention (system calls). |
| 5. | Here the processes need to ensure that they are not writing to the same location simultaneously. | It is useful for sharing small amounts of data as conflicts need not to be resolved. |
| 6. | Faster communication strategy. | Relatively slower communication strategy |

# Ways of Inter-Process Communication

**Synchronous IPC**

- The processes of sending & receiving messages are synchronized with each message.
- Both the send & receive are blocking operations.
- Once a send is issued to the sending process then it is blocked until the equivalent receive is issued.
- Similarly, when a receive is issued through a process, then it blocked until a message appears.

# Ways of Inter-Process Communication

**Asynchronous IPC**

- The send operation is non-blocking because the process of sending will be allowed to continue once the message has been copied towards a local buffer & the transmission of the message will continue in parallel throughout the sending process.
- The receive operation includes blocking & non-blocking variants.

# Methods in Interprocess Communication

- There are several different ways to implement IPC
- IPC is set of programming interfaces, used by programs to communicate between series of processes.
- This allows running programs concurrently in an Operating System
  - **Pipes (Same Process)**
  - **Names Pipes (Different Processes)**
  - **Message Queuing**
  - **Sockets**
  - **Semaphores**
  - **Shared memory**

# Methods in Interprocess Communication

**Pipes:** This allows flow of data in one direction only. Analogous to simplex systems (Keyboard). Data from the output is usually buffered until input process receives it which must have a common origin.

**Named pipes:** This is a pipe with a specific name it can be used in processes that don't have a shared common process origin.

**Sockets :** This method is mostly used to communicate over a network between a client and a server. It allows for a standard connection which is computer and OS independent.

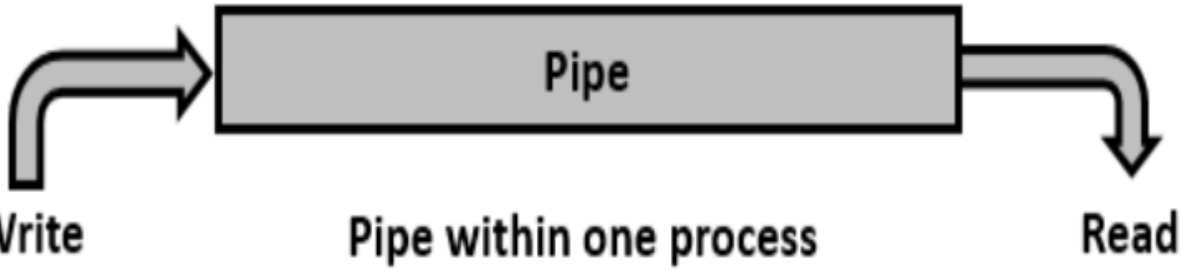
**Semaphores :** This is used in solving problems associated with synchronization and to avoid race condition. These are integer values which are greater than or equal to 0.

**Shared memory :** This allows the interchange of data through a defined area of memory. Semaphore values have to be obtained before data can get access to shared memory.

# IPC using Pipes

- Pipe is a communication medium between two or more related or interrelated processes.
- It can be either within one process or a communication between the child and the parent processes.
- Communication can also be multi-level such as communication between the parent, the child and the grand-child, etc.
- Communication is achieved by one process writing into the pipe and other reading from the pipe.
- To achieve the pipe system call, create two files, one to write into the file and another to read from the file.

# Pipes in IPC




```
#include<unistd.h>

int pipe(int pipedes[2]);
```

This system call would create a pipe for one-way communication i.e., it creates two descriptors, first one is connected to read from the pipe and other one is connected to write into the pipe.

```c
#include<unistd.h>

int main() {
   int pipefds[2];
   int returnstatus;
   char writemessages[2][20]={"Hi", "Hello"};
   char readmessage[20];
   returnstatus = pipe(pipefds);

   if (returnstatus == -1) {
      printf("Unable to create pipe\n");
      return 1;
   }

   printf("Writing to pipe - Message 1 is %s\n", writemessages[0]);
   write(pipefds[1], writemessages[0], sizeof(writemessages[0]));
   read(pipefds[0], readmessage, sizeof(readmessage));
   printf("Reading from pipe – Message 1 is %s\n", readmessage);
   printf("Writing to pipe - Message 2 is %s\n", writemessages[0]);
   write(pipefds[1], writemessages[1], sizeof(writemessages[0]));
   read(pipefds[0], readmessage, sizeof(readmessage));
   printf("Reading from pipe – Message 2 is %s\n", readmessage);
   return 0;
}
```

# Pipes

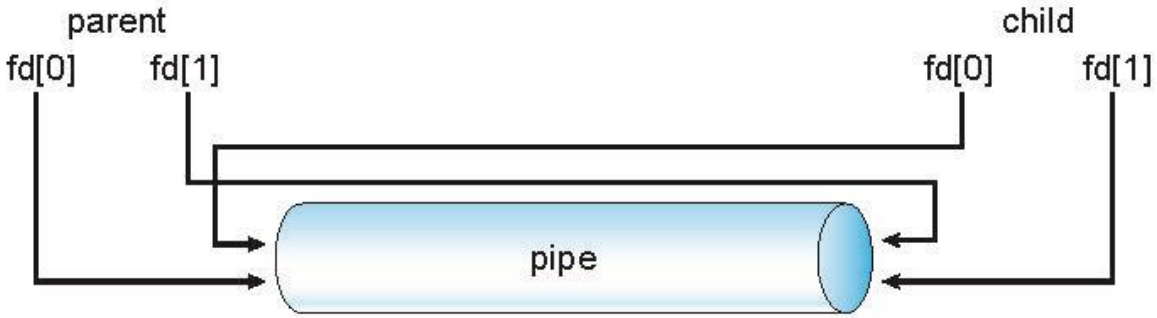
- Acts as a conduit allowing two processes to communicate
- **Ordinary pipes** – cannot be accessed from outside the process that created it. Typically, a parent process creates a pipe and uses it to communicate with a child process that it created.
- **Named pipes** – can be accessed without a parent-child relationship.

# Unnamed and Named Pipes

- There are two types of pipes
- Unnamed pipes
  - Used only with related processes
  - Parent/child
  - Child/child
  - The pipe exists only as long as the processes using it are alive
- Named pipes
  - Actually exist as directory entries
  - Have file access permissions
  - Can be used by unrelated processes

# Ordinary Pipes

- Ordinary Pipes allow communication in standard producer-consumer style
- Producer writes to one end (the **write-end** of the pipe)
- Consumer reads from the other end (the **read-end** of the pipe)
- Ordinary pipes are therefore unidirectional
- Require parent-child relationship between communicating processes




- Windows calls these **anonymous pipes**
- See Unix and Windows code samples in textbook

# Named Pipes

- In computing, **a named pipe** (also known as a FIFO) is one of the methods for inter-process communication.
- It is an extension to the traditional pipe concept on Unix.
- A traditional pipe is “unnamed” and lasts only as long as the process.
- A named pipe, however, can last as long as the system is up, beyond the life of the process. It can be deleted if no longer used.
- Named Pipes are more powerful than ordinary pipes
- Communication is bidirectional
- No parent-child relationship is necessary between the communicating processes
- Provided on both UNIX and Windows systems

# Named Pipe

- To create a named pipe, the command is:
  - **mkfifo <pipe-name>**

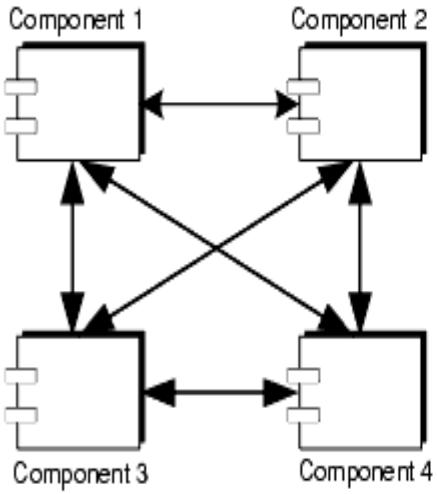
- Another way to create a FIFO named pipe is to use this command:
  - **mknod p <pipe-name>**

# IPC using Message Queues

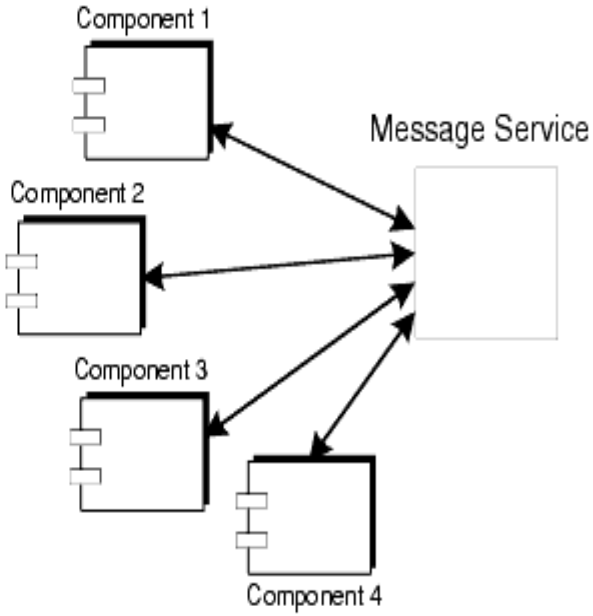
- A **message queue** is an inter-process communication (IPC) mechanism that allows processes to exchange data in the form of messages between two processes.
- It allows processes to communicate asynchronously by sending messages to each other where the messages are stored in a queue, waiting to be processed, and are deleted after being processed.
- The message queue is a buffer that is used in non-shared memory environments, where tasks communicate by passing messages to each other rather than by accessing shared variables.
- Messages can be of any size.

# Messaging Methods

**Peer to Peer Messaging**

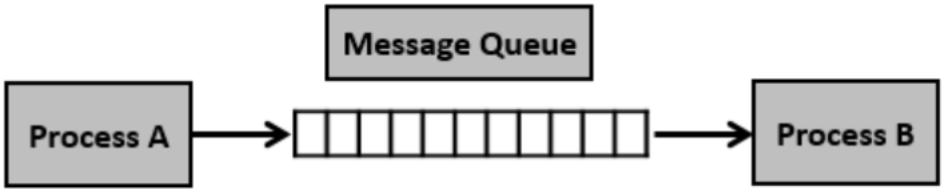



**Centralized Messaging**

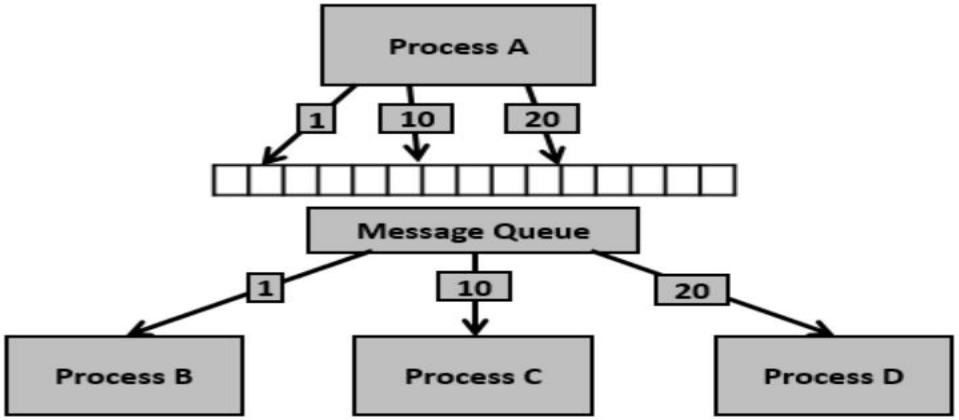


# Communication using message queues – 2 types

- Writing by one process and reading by another process.



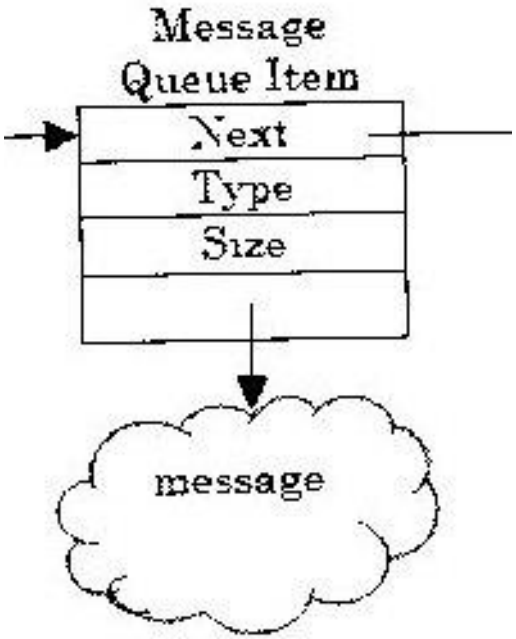

- Writing by one process with different data packets and reading by multiple processes, i.e., as per message type



# Attributes of Message Queues

Every message on a queue has the following attributes:

- Message type
- Message length (length of data part of message)
- Message data (if length is greater than 0)



# *msg* structure

```
struct   msg  {
 struct  msg    *msg_next;  /* pointer to next message on q */
 long           msg_type;  /*  message type */
 ushort          msg_ts;     /*  message text size */
 short          msg_spot;  /* message text map address */
};
```

# Steps in Communication using MsgQueue

To perform communication using message queues, following are the steps −

**Step 1** − Create a message queue or connect to an already existing message queue (msgget())

**Step 2** − Write into message queue (msgsnd())

**Step 3** − Read from the message queue (msgrcv())

**Step 4** − Perform control operations on the message queue (msgctl())

# Functions of Message Queue

- There are four important functions that we will use in the programs to achieve IPC using message queues.

**1. int msgget (key_t key, int msgflg);**

We use the msgget function to create and access a message queue. It takes two parameters.

- The first parameter is a key that names a message queue in the system.
- The second parameter is used to assign permission to the message queue and is ORed with IPC_CREAT to create the queue if it doesn't already exist. If the queue already exists, then IPC_CREAT is ignored.
- On success, the msgget function returns a positive number which is the queue identifier, while on failure, it returns -1.

# Functions of Message Queue

**2. int msgsnd (int msqid, const void *msg_ptr, size_t msg_sz, int msgflg);**

- This function allows us to add a message to the message queue.
- The first parameter (msgid) is the message queue identifier returned by the msgget function.
- The second parameter is the pointer to the message to be sent, which must start with a long int type.
- The third parameter is the size of the message. It must not include the long int message type.
- The fourth and final parameter controls what happens if the message queue is full or the system limit on queued messages is reached. The function on success returns 0 and place the copy of message data on the message queue. On failure, it returns -1.

# Functions of Message Queue

**3. int msgrcv (int msqid, void *msg_ptr, size_t msg_sz, long int msgtype, int msgflg);**

- This function retrieves messages from a message queue.
- The first parameter (msgid) is the message queue identifier returned by the msgget function.
- The second parameter is the pointer to the message to be received, which must start with a long int type.
- The third parameter is the size of the message.
- The fourth parameter allows implementing priority. 0 value means to receive the messages in the order in which they were sent, and non zero means receive the message with a specific message type.
- The final parameter controls what happens if the message queue is full or the system limit on queued messages is reached. The function on success returns 0 and place the copy of message data on the message queue. On failure, it returns -1.

# Functions of Message Queue

- **4. int msgctl (int msqid, int command, struct msqid_ds *buf);**
  - The final function is msgctl, which is the control function.
  - The first parameter is the identifier returned by the msgget function.
  - The second parameter can have one out of the below three values.

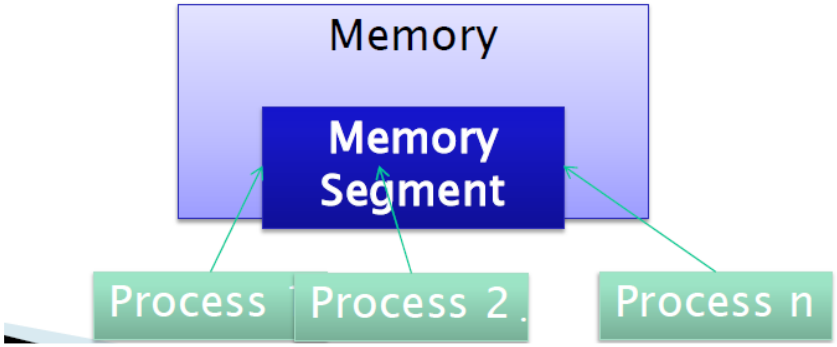
| | |
|---|---|
| IPC_STAT | It sets the data in the msqid_ds structure to reflect the values associated with the message queue. |
| IPC_SET | If the process has permission to do so, this sets the values associated with the message queue to those provided in the msqid_ds data structure. |
| IPC_RMID | It deletes the message queue. |

- The msgctl function returns 0 on success and -1 on error.

# Shared Memory Model

**Shared memory (SHM) :**

A method of inter-process communication (IPC) whereby 2 or more processes share a single chunk of memory to communicate.



# Shared Memory

Steps of Shared Memory IPC

1. Creating the segment and connecting
2. Getting a pointer to the segment
3. Reading and Writing
4. Detaching from and deleting segments

# Shared Memory

1. Creating the segment and connecting System V IPC
   A shared memory segment is ‘created’ and ‘connected to’ via the shmget() call

- int shmget(key_t key, size_t size, int shmflg);
  - The key argument should be created using ftok().
  - The size, is the size in bytes of the shared memory segment.
  - The shmflg should be set to the permissions of the segment bitwise-ORd with IPC_CREAT if you want to create the segment, but can be 0 otherwise.
  - Upon successful completion, shmget() returns an identifier for the shared memory segment.

# Shared Memory

2.Creating the segment ad connecting

Here's an example call that creates a 1K segment with 644 permissions (rw-r--r--)

```
key_t key;
int shmid;
key = ftok("/home/somefile3", 'R');
shmid = shmget(key, 1024, 0644 | IPC_CREAT);
```

A Sticky bit is a permission bit that is set on a file or a directory that lets only the owner of the file/directory or the root user to delete or rename the file.

# Shared Memory

- Getting a pointer to the segment
- The shared memory segment must be attached using shmat() before its used.

```
void *shmat(int shmid, void *shmaddr, int shmflg);
```

- shmid is the shared memory ID from the shmget() call.
- shmaddr, which you can use to tell shmat() which specific address to use.
- When set it to 0, the OS will decide the address. shmflg can be set to SHM_RDONLY if you only want to read from it, 0 otherwise.

# Shared Memory

Here's a more complete example of how to get a pointer to a shared memory segment:

```
key_t key;
int shmid;
char *data;

key = ftok("/home/beej/somefile3", 'R');
shmid = shmget(key, 1024, 0644 | IPC_CREAT);
data = shmat(shmid, (void *)0, 0);
```

# Reading and Writing into Shared Memory

The *data* pointer from the above example is a char pointer. Thus it reads chars from it.

lets say the 1K shared memory segment contains a null-terminated string.

It can be printed like this:

```
printf("shared contents: %s\n", data);
```

And we could store something in it as easily as this:

```
printf("Enter a string: ");
gets(data);
```

# Detaching from and deleting segments from Shared Memory

When you're done with the shared memory segment your program should detach itself from it using the **shmdt()** call:

```
int shmdt(void *shmaddr);
```

The only argument, *shmaddr*, is the address you got from **shmat()**.

The function returns -1 on error, 0 on success.

# Detaching from and deleting segments from Shared Memory

**Remember!** When you detach from the segment, it isn't destroyed. Nor is it removed when *everyone* detaches from it.

You have to specifically destroy it using a call to **shmctl()**

```
shmctl(shmid, IPC_RMID, NULL);
```

The above call deletes the shared memory segment, assuming no one else is attached to it.

# Pros and Cons Shared Memory

## Pros

- Fast bidirectional communication among any number of processes
- Saves Resources

## Cons

- Needs concurrency control (leads to data inconsistencies like 'Lost update')
- Lack of data protection from Operating System (OS)

# Lab Exercise on IPC

Q1. A C program for IPC using shared memory

https://dextutor.com/program-for-ipc-using-shared-memory/

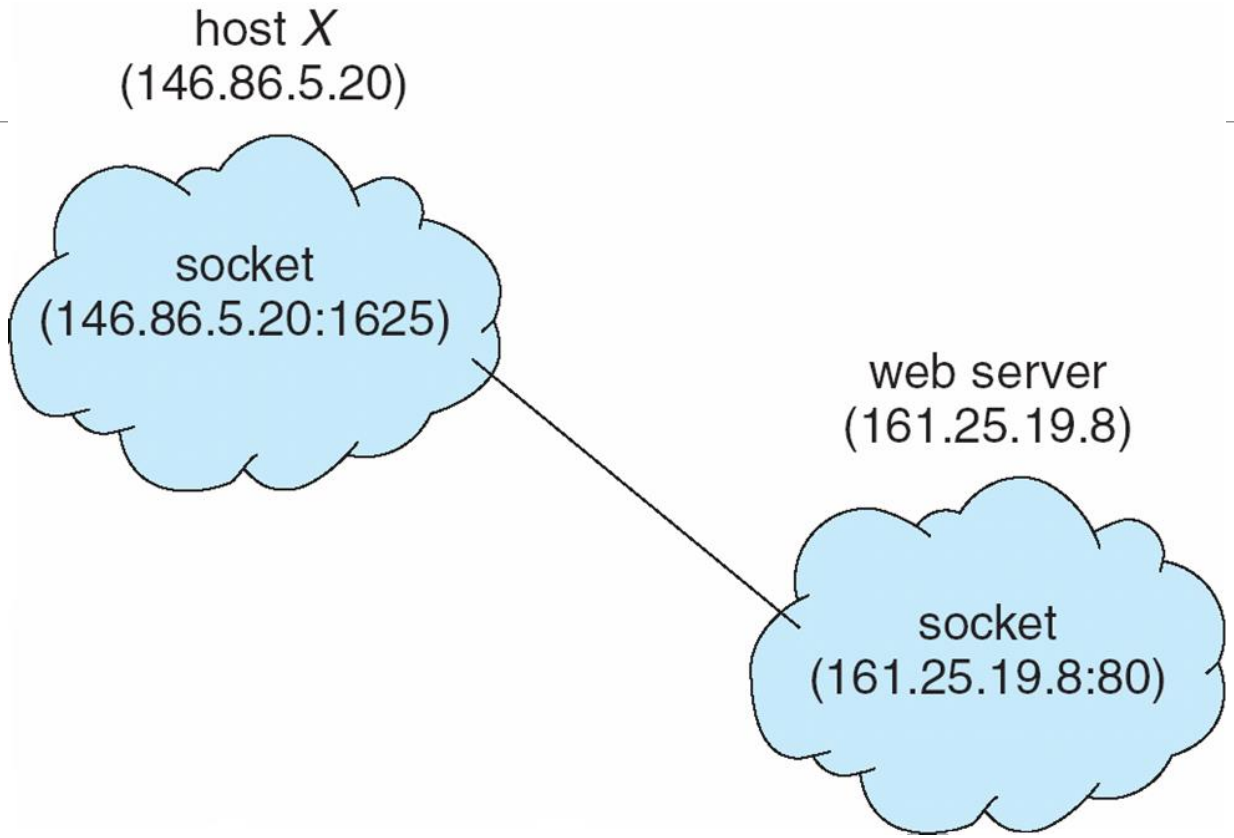
Q2. A C program for IPC using Message Queues

https://www.javatpoint.com/ipc-using-message-queues

# Sockets

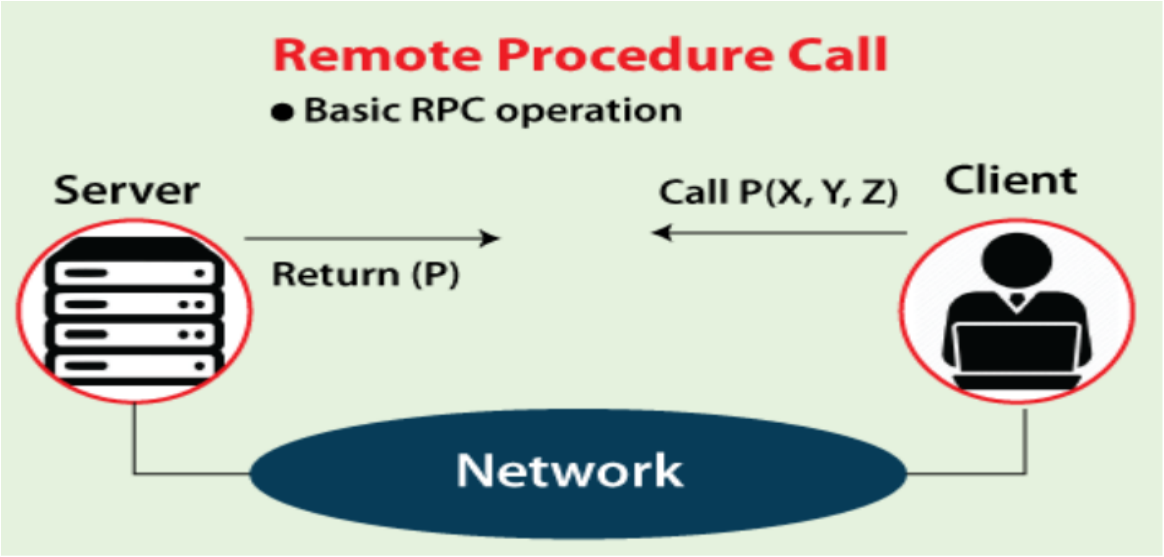
- A **socket** is defined as an endpoint for communication
- Concatenation of IP address and **port** – a number included at start of message packet to differentiate network services on a host
- The socket **161.25.19.8:1625** refers to port **1625** on host **161.25.19.8**
- Communication consists between a pair of sockets
- All ports below 1024 are ***well known***, used for standard services
- Special IP address 127.0.0.1 (**loopback**) to refer to system on which process is running

# Socket Communication

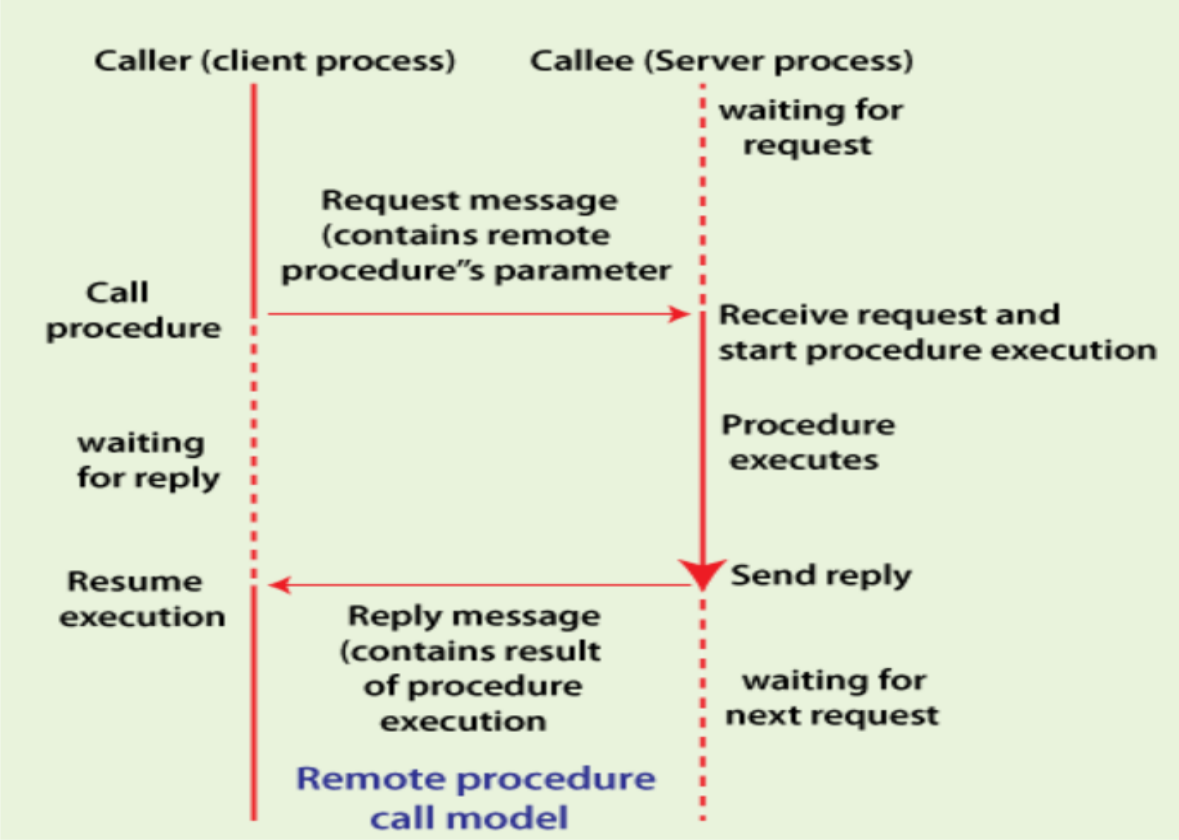


# Remote Procedure Calls (RPC)

- Remote Procedure Call or RPC is a powerful technique for constructing distributed, client-server-based applications.
- It is also known as a function call or a subroutine call.
- A remote procedure call is when a computer program causes a procedure to execute in a different address space, coded as a local procedure call, without the programmer explicitly stating the details for the remote interaction.



# Remote Procedure Calls in OS



# IPC System Calls

| Functionality | Message Queue | Semaphore | Shared |
|---|---|---|---|
| Allocate IPC | *Msgget* | *semget* | *shmget* |
| Access IPC | *msgsnd msgrcv* | *semop* | *shmat shmdt* |
| IPC Control | *msgctl* | *semctl* | *shmctl* |

# Questions

1. Which of the following are TRUE for direct communication?

a) A communication link can be associated with N number of processes
b) A communication link is associated with exactly two processes
c) Exactly N/2 links exist between each pair of processes(N = max. number of processes supported by system)
d) Exactly two link exists between each pair of processes

2. Which of the following is the fastest IPC mechanism?

a) Pipe
b) Message queue
c) Socket
d) Shared memory

10. The Zero Capacity queue _____

a) is referred to as a message system with buffering

b) is referred to as a message system with no buffering

c) is referred to as a link

d) none of the mentioned

# End of Chapter 3